CES-AI Buddy-POC-Code Graph-Example

Development Reference · AI-Generated Technical Response

MODE

AppData+Feature

QUESTION

Track users online now. Create new table, controller, view. Donot change any existing view, model , controller. Show online users in grid in admin menu

Auto-detected: view=CES\Views\Shared\EditorTemplates\String.cshtml · controller=Buses · action=(none) · table=Users

DB

```
# DB Layer - Online Users Tracking

## CHANGE 1: DB - schema.sql
OP: NEW_FILE
ANCHOR (omit for NEW_FILE):
<<<
>>>
CODE:
<<<
-- ============================================
-- Online Users Tracking Table
-- ============================================
CREATE TABLE [SBCES].[OnlineUsers](
■[OnlineUserId] [int] IDENTITY(1,1) NOT NULL,
■[UserID] [int] NOT NULL,
■[SessionId] [varchar](255) NOT NULL,
■[LastActivityTime] [datetime] NOT NULL,
■[IPAddress] [varchar](50) NULL,
■[UserAgent] [varchar](500) NULL,
 CONSTRAINT [PK_OnlineUsers] PRIMARY KEY CLUSTERED
(
■[OnlineUserId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS =
ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY],
 CONSTRAINT [FK_OnlineUsers_Users] FOREIGN KEY([UserID])
■REFERENCES [SBCES].[Users] ([UserID])
) ON [PRIMARY]
GO

CREATE NONCLUSTERED INDEX [IX_OnlineUsers_UserID] ON [SBCES].[OnlineUsers]
(
■[UserID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING =
OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY =
OFF) ON [PRIMARY]
GO

CREATE NONCLUSTERED INDEX [IX_OnlineUsers_SessionId] ON [SBCES].[OnlineUsers]
(
■[SessionId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING =
OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY =
OFF) ON [PRIMARY]
GO

CREATE NONCLUSTERED INDEX [IX_OnlineUsers_LastActivityTime] ON [SBCES].[OnlineUsers]
(
```

```
■[LastActivityTime] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, DROP_EXISTING =
OFF, ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY =
OFF) ON [PRIMARY]
GO
>>>
WHY: Create OnlineUsers table with IDENTITY PK per HAS_IDENTITY convention, explicit FK to
Users table, indexes for efficient querying by UserID/SessionId/LastActivityTime

## CHANGE 2: DB - rollback.sql
OP: NEW_FILE
ANCHOR (omit for NEW_FILE):
<<<
>>>
CODE:
<<<
-- ============================================
-- Rollback Script for Online Users Tracking
-- ============================================
DROP INDEX IF EXISTS [IX_OnlineUsers_LastActivityTime] ON [SBCES].[OnlineUsers]
GO

DROP INDEX IF EXISTS [IX_OnlineUsers_SessionId] ON [SBCES].[OnlineUsers]
GO

DROP INDEX IF EXISTS [IX_OnlineUsers_UserID] ON [SBCES].[OnlineUsers]
GO

DROP TABLE IF EXISTS [SBCES].[OnlineUsers]
GO
>>>
WHY: Rollback script to remove OnlineUsers table and indexes in reverse order of creation

---

## ALSO NEEDS (Model, answered in that layer's own section)
OnlineUser model class

## ALSO NEEDS (Controller, answered in that layer's own section)
OnlineUsersController with Read action for Kendo grid, background logic to track user activity

## ALSO NEEDS (View, answered in that layer's own section)
Index.cshtml view with Kendo Grid for displaying online users

## ALSO NEEDS (JavaScript, answered in that layer's own section)
Grid configuration and auto-refresh logic
```

CONTROLLER

```
# CONTROLLER Layer - Online Users Tracking

## CHANGE 1: Controller - Controllers\Admin\OnlineUsersController.cs
OP: NEW_FILE
ANCHOR (omit for NEW_FILE):
<<<
>>>
CODE:
<<<
using BCES.Data;
using Microsoft.AspNetCore.Mvc;
using System.Data;
using BCES.Controllers.Base;
using BCES.Models.Admin;
using Dapper;
```

```
using Kendo.Mvc.UI;
using Kendo.Mvc.Extensions;

namespace BCES.Controllers.Admin
{
    public class OnlineUsersController : BaseController
    {
        private readonly DapperContext _db;
        private readonly IDbConnection _dbConnection;
        private readonly IHttpContextAccessor _httpContextAccessor;

        public OnlineUsersController(DapperContext dapper, IHttpContextAccessor
httpContextAccessor) : base(dapper, httpContextAccessor)
        {
            _db = dapper;
            _dbConnection = _db.CreateConnection();
            _httpContextAccessor = httpContextAccessor;
        }

        public IActionResult Index()
        {
            if (ViewBag.RoleId != 1)
            {
                return View("~/Views/Shared/Unauthorized.cshtml");
            }
            return View();
        }

        public async Task<IActionResult> Read([DataSourceRequest] DataSourceRequest request)
        {
            try
            {
                if (ViewBag.RoleId != 1)
                {
                    return StatusCode(403, "Unauthorized access");
                }

                await CleanupStaleSessionsAsync();

                var query = @"
                    SELECT
                        ou.[OnlineUserId],
                        ou.[UserID],
                        u.[UserName],
                        u.[UserADId],
                        r.[RoleName],
                        ou.[SessionId],
                        ou.[LastActivityTime],
                        ou.[IPAddress],
                        ou.[UserAgent]
                    FROM [SBCES].[OnlineUsers] ou
                    INNER JOIN [SBCES].[Users] u ON ou.[UserID] = u.[UserID]
                    LEFT JOIN [SBCES].[Roles] r ON u.[RoleId] = r.[RoleId]
                    ORDER BY ou.[LastActivityTime] DESC";

                var onlineUsers = await _dbConnection.QueryAsync<OnlineUserViewModel>(query);
                return Json(onlineUsers.ToDataSourceResult(request));
            }
            catch (Exception ex)
            {
                return StatusCode(500, ex.Message);
            }
        }

        [HttpPost]
        public async Task<IActionResult> UpdateActivity()
        {
```

```
            try
            {
                var sessionId = _httpContextAccessor.HttpContext.Session.Id;
                var userADId = _httpContextAccessor.HttpContext.User.Identity.Name;

                if (string.IsNullOrEmpty(userADId))
                {
                    return StatusCode(401, "User not authenticated");
                }

                string[] temp = userADId.Split('\\');
                userADId = temp.Length > 1 ? temp[1] : userADId;

                var userId = await _dbConnection.QueryFirstOrDefaultAsync<int?>(
                    @"SELECT [UserID] FROM [SBCES].[Users] WHERE [UserADId] = @UserADId",
                    new { UserADId = userADId });

                if (!userId.HasValue)
                {
                    return StatusCode(404, "User not found");
                }

                var ipAddress =
_httpContextAccessor.HttpContext.Connection.RemoteIpAddress?.ToString();
                var userAgent =
_httpContextAccessor.HttpContext.Request.Headers["User-Agent"].ToString();

                var existingRecord = await _dbConnection.QueryFirstOrDefaultAsync<int?>(
                    @"SELECT [OnlineUserId] FROM [SBCES].[OnlineUsers]
                      WHERE [SessionId] = @SessionId",
                    new { SessionId = sessionId });

                if (existingRecord.HasValue)
                {
                    await _dbConnection.ExecuteAsync(
                        @"UPDATE [SBCES].[OnlineUsers]
                          SET [LastActivityTime] = GETDATE(),
                              [IPAddress] = @IPAddress,
                              [UserAgent] = @UserAgent
                          WHERE [SessionId] = @SessionId",
                        new { SessionId = sessionId, IPAddress = ipAddress, UserAgent =
userAgent });
                }
                else
                {
                    await _dbConnection.ExecuteAsync(
                        @"INSERT INTO [SBCES].[OnlineUsers]
                          ([UserID], [SessionId], [LastActivityTime], [IPAddress], [UserAgent])
                          VALUES (@UserID, @SessionId, GETDATE(), @IPAddress, @UserAgent)",
                        new { UserID = userId.Value, SessionId = sessionId, IPAddress =
ipAddress, UserAgent = userAgent });
                }

                return Ok();
            }
            catch (Exception ex)
            {
                return StatusCode(500, ex.Message);
            }
        }

        private async Task CleanupStaleSessionsAsync()
        {
            try
            {
                var timeoutMinutes = await _dbConnection.QueryFirstOrDefaultAsync<int?>(
                    @"SELECT CAST([Value] AS INT) FROM [SBCES].[CesSettings]
```

```
                      WHERE [Parameter] = 'SESSIONTIMEOUTMINUTES'");

                var timeout = timeoutMinutes ?? 30;

                await _dbConnection.ExecuteAsync(
                    @"DELETE FROM [SBCES].[OnlineUsers]
                      WHERE DATEADD(MINUTE, @Timeout, [LastActivityTime]) < GETDATE()",
                    new { Timeout = timeout });
            }
            catch (Exception ex)
            {
                // Log error but don't fail the request
            }
        }
    }
}
>>>
WHY: New OnlineUsersController with Read action for Kendo grid, UpdateActivity endpoint for
tracking user sessions, CleanupStaleSessionsAsync helper using SESSIONTIMEOUTMINUTES from
CesSettings, inherits BaseController per convention

## CHANGE 2: Model - Models\Admin\OnlineUserViewModel.cs
OP: NEW_FILE
ANCHOR (omit for NEW_FILE):
<<<
>>>
CODE:
<<<
using System;

namespace BCES.Models.Admin
{
    public class OnlineUserViewModel
    {
        public int OnlineUserId { get; set; }
        public int UserID { get; set; }
        public string UserName { get; set; }
        public string UserADId { get; set; }
        public string RoleName { get; set; }
        public string SessionId { get; set; }
        public DateTime LastActivityTime { get; set; }
        public string IPAddress { get; set; }
        public string UserAgent { get; set; }
    }
}
>>>
WHY: ViewModel for OnlineUsers grid with joined Users/Roles data per Read action query

---

## ALSO NEEDS (View, answered in that layer's own section)
Views\Admin\OnlineUsers\Index.cshtml with Kendo Grid configured for OnlineUserViewModel columns

## ALSO NEEDS (JavaScript, answered in that layer's own section)
Auto-refresh logic calling UpdateActivity endpoint periodically, grid refresh timer

## ALSO NEEDS (Shared Layout, answered in that layer's own section)
Admin menu link to /OnlineUsers/Index
```

JS

```
# JS Layer - Online Users Tracking

## LAYER SCOPE ANALYSIS
```

```
This call is scoped to **standalone .js files only** (functions that live in their own file,
not inline `<script>` inside a .cshtml).

The VIEW layer response shows all JavaScript for this feature is **inline** within
`Views\Admin\OnlineUsers\Index.cshtml` (the `<script>` block containing `trackUserActivity()`,
grid auto-refresh logic, and activity event handlers).

Per the layer scope rule: "functions that live in their own file, not inline `<script>` inside
a .cshtml - that belongs to the VIEW layer's own call" - this inline script is **not in
scope** for this JS layer call.

## FINDINGS

**No standalone .js file changes needed for this feature.**

All JavaScript required for the online users tracking feature was implemented inline in the
view (Views\Admin\OnlineUsers\Index.cshtml) per the VIEW layer's response, which is the
correct pattern for view-specific logic in this codebase.

The inline script handles:
- Grid auto-refresh (30-second interval via `setInterval`)
- Activity tracking via `trackUserActivity()` function
- Throttled AJAX calls to `OnlineUsers/UpdateActivity` endpoint
- Event listeners for user interaction (mousemove, keydown, click)

This follows the same pattern seen in CONTEXT for other admin grids where view-specific
JavaScript lives inline rather than in separate .js files.

---

## ALSO NEEDS
None - all JavaScript requirements for this feature are satisfied by the inline script in the
VIEW layer's Index.cshtml.
```

VIEW

```
# VIEW Layer - Online Users Tracking

## CHANGE 1: View - Views\Admin\OnlineUsers\Index.cshtml
OP: NEW_FILE
ANCHOR (omit for NEW_FILE):
<<<
>>>
CODE:
<<<
@{
    ViewData["Title"] = "Online Users";
}

<div class="container-fluid mt-4">
    <div class="row">
        <div class="col-12">
            <h2>Online Users</h2>
            <p class="text-muted">Real-time view of currently active users in the system</p>
        </div>
    </div>

    <div class="row mt-3">
        <div class="col-12">
            @(Html.Kendo().Grid<BCES.Models.Admin.OnlineUserViewModel>()
                .Name("OnlineUsersGrid")
                .Columns(columns =>
                {
                    columns.Bound(p => p.UserName).Title("User Name").Width(150);
```

```
                    columns.Bound(p => p.UserADId).Title("AD ID").Width(150);
                    columns.Bound(p => p.RoleName).Title("Role").Width(120);
                    columns.Bound(p => p.LastActivityTime).Title("Last
Activity").Format("{0:yyyy-MM-dd HH:mm:ss}").Width(180);
                    columns.Bound(p => p.IPAddress).Title("IP Address").Width(130);
                    columns.Bound(p => p.UserAgent).Title("Browser/Device").Width(250);
                    columns.Bound(p => p.SessionId).Title("Session
ID").Width(200).Hidden(true);
                })
                .Pageable(pageable => pageable
                    .Refresh(true)
                    .PageSizes(new int[] { 10, 25, 50, 100 })
                    .ButtonCount(5))
                .Sortable()
                .Scrollable()
                .Filterable()
                .DataSource(dataSource => dataSource
                    .Ajax()
                    .PageSize(25)
                    .Read(read => read.Action("Read", "OnlineUsers"))
                )
                .HtmlAttributes(new { style = "height:600px;" })
            )
        </div>
    </div>

    <div class="row mt-3">
        <div class="col-12">
            <div class="alert alert-info">
                <strong>Note:</strong> This grid auto-refreshes every 30 seconds. Users are
considered offline after @ViewBag.SessionTimeoutMinutes minutes of inactivity.
            </div>
        </div>
    </div>
</div>

<script>
    $(document).ready(function () {
        var grid = $("#OnlineUsersGrid").data("kendoGrid");

        // Auto-refresh every 30 seconds
        setInterval(function () {
            grid.dataSource.read();
        }, 30000);

        // Track current user's activity
        trackUserActivity();

        // Track activity on any interaction
        $(document).on("mousemove keydown click", function () {
            trackUserActivity();
        });
    });

    var lastActivityUpdate = 0;
    var activityThrottleMs = 60000; // Update every 60 seconds max

    function trackUserActivity() {
        var now = Date.now();
        if (now - lastActivityUpdate < activityThrottleMs) {
            return; // Throttle updates
        }
        lastActivityUpdate = now;

        $.ajax({
            url: '@Url.Action("UpdateActivity", "OnlineUsers")',
            type: 'POST',
```

```
            headers: {
                'RequestVerificationToken': $('input[name="__RequestVerificationToken"]').val()
            },
            success: function () {
                console.log('Activity tracked');
            },
            error: function (xhr) {
                console.error('Failed to track activity:', xhr.responseText);
            }
        });
    }
</script>
>>>
WHY: New Index view with Kendo Grid displaying online users, auto-refresh every 30 seconds,
inline script for activity tracking via UpdateActivity endpoint established in Controller layer

---

## ALSO NEEDS (JavaScript, answered in that layer's own section)
None - all JavaScript is inline in the view per LAYER SCOPE

---

## 1.a Kendo/Telerik controls used or added
- **Control**: `Html.Kendo().Grid<BCES.Models.Admin.OnlineUserViewModel>()`
- **Helper chain**: `.Name("OnlineUsersGrid")` → `.Columns()` → `.Pageable()` → `.Sortable()`
→ `.Scrollable()` → `.Filterable()` → `.DataSource()` with `.Ajax()` → `.Read(read =>
read.Action("Read", "OnlineUsers"))`
- **File**: Views\Admin\OnlineUsers\Index.cshtml

## 1.b Shared/EditorTemplates/Partials used
None - this view does not reference any shared partials, editor templates, or layout overrides
beyond the default _Layout.cshtml inheritance (implicit via ASP.NET Core MVC convention, not
explicitly referenced in this file's code).

**USED_BY**: N/A (new file, no existing consumers)

**Regression risk**: None - this is a new standalone view under a new controller
(OnlineUsersController established in Controller layer); it does not modify any existing view,
shared partial, or editor template per the requirement "Donot change any existing view, model,
controller."
```

Source validation: 0/0 code citations verified against the real files.

Retrieval notes

- VIEW MATCH UNCERTAIN: no file was named exactly "existing" (or "_existing") - the closest real view name found was "string" (CES\Views\Shared\EditorTemplates\String.cshtml), used as a best guess. If this isn't the file you meant, provide the exact path via the View hint instead.
- NEW ACTION (not yet in codebase): Buses.(new action - not yet named). No existing implementation of this action to match, but the rest of BusesController is included below as a real anchor point, plus BaseController/DapperContext for DI and error-handling conventions.

Generated October 28, 2026 at 02:48 PM